



ABSTRACT FACTORY PATTERN

(Creational Design Pattern – Nâng cao)

1 Mục tiêu bài học

- Hiểu sự khác nhau giữa Factory và Abstract Factory
- Nhận diện bài toán cần tạo nhiều object liên quan theo cùng ngữ cảnh
- Áp dụng Abstract Factory trong hệ thống Order Management
- Kết hợp Strategy + Factory + DIP
- Hiểu vai trò Abstract Factory trong Clean Architecture & mở rộng hệ thống

2 Vấn đề: Hệ thống đa thị trường

Thành phần	VN Market	US Market
Payment	VNPay	PayPal
Tax	VAT	Sales Tax
Invoice	VAT Invoice	US Invoice
Currency	VND	USD

3 Vấn đề (Tiếp)

? Câu hỏi thiết kế:

- Khi đổi Market (VN → US), có phải sửa OrderService không?

→ Nếu CÓ → thiết kế kém.

→ Cần cơ chế đổi toàn bộ "hệ sinh thái" object một cách linh hoạt.

4 Vì sao Factory thường KHÔNG đủ?

✗ Dùng Simple Factory rời rạc:

- CreatePayment(); CreateTax(); CreateInvoice();

📌 Vấn đề:

- Các object không được đảm bảo cùng hệ.
- Dễ mix sai: VNPay + US Tax (Râu ông nọ cắm cằm bà kia).
- Business phải gọi nhiều factory khác nhau.

→ Cần tạo cả một họ object đồng bộ.

5 Định nghĩa Abstract Factory

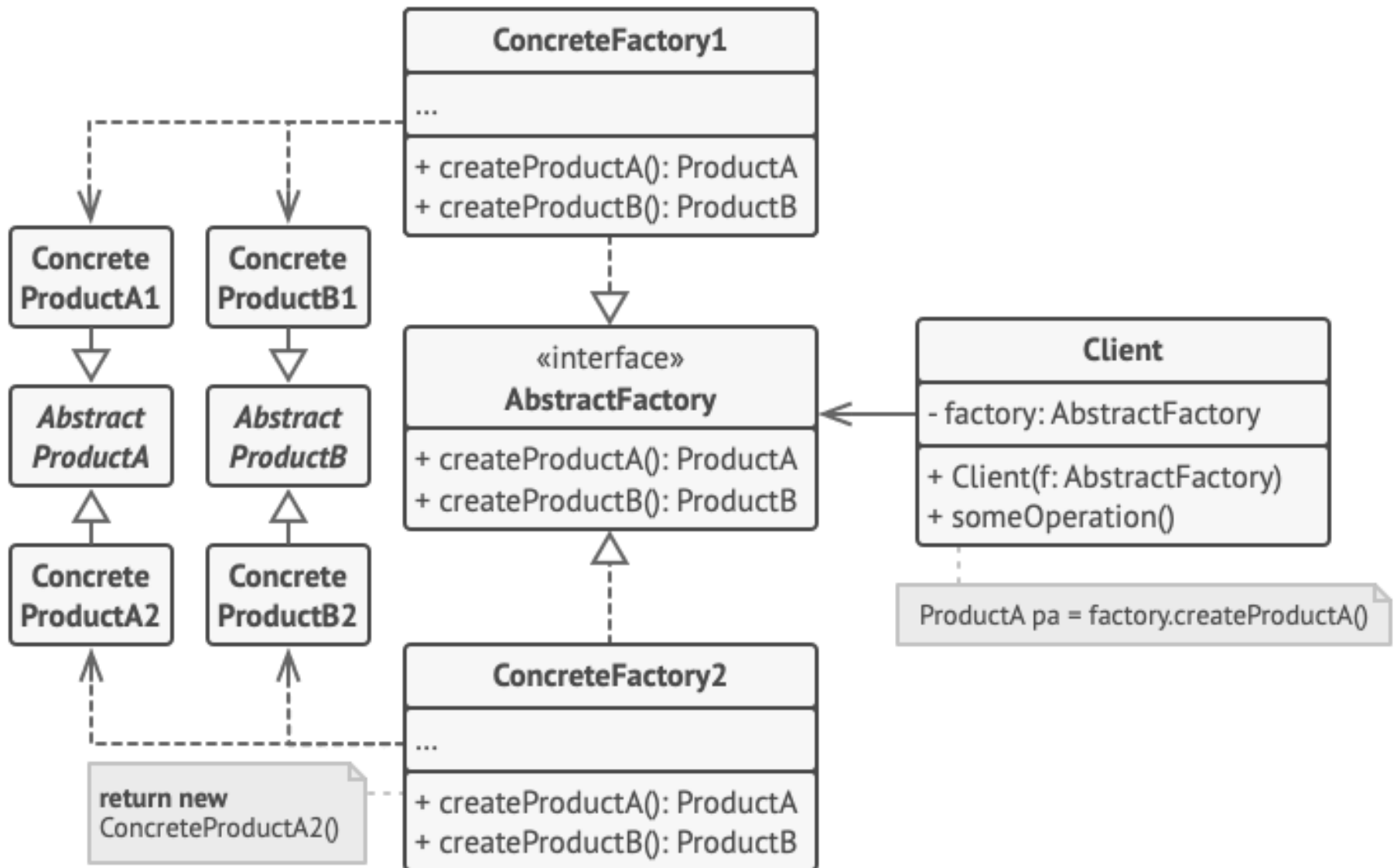
Abstract Factory Pattern:

- Cung cấp interface để tạo một họ các object liên quan mà không cần chỉ ra class cụ thể của chúng.

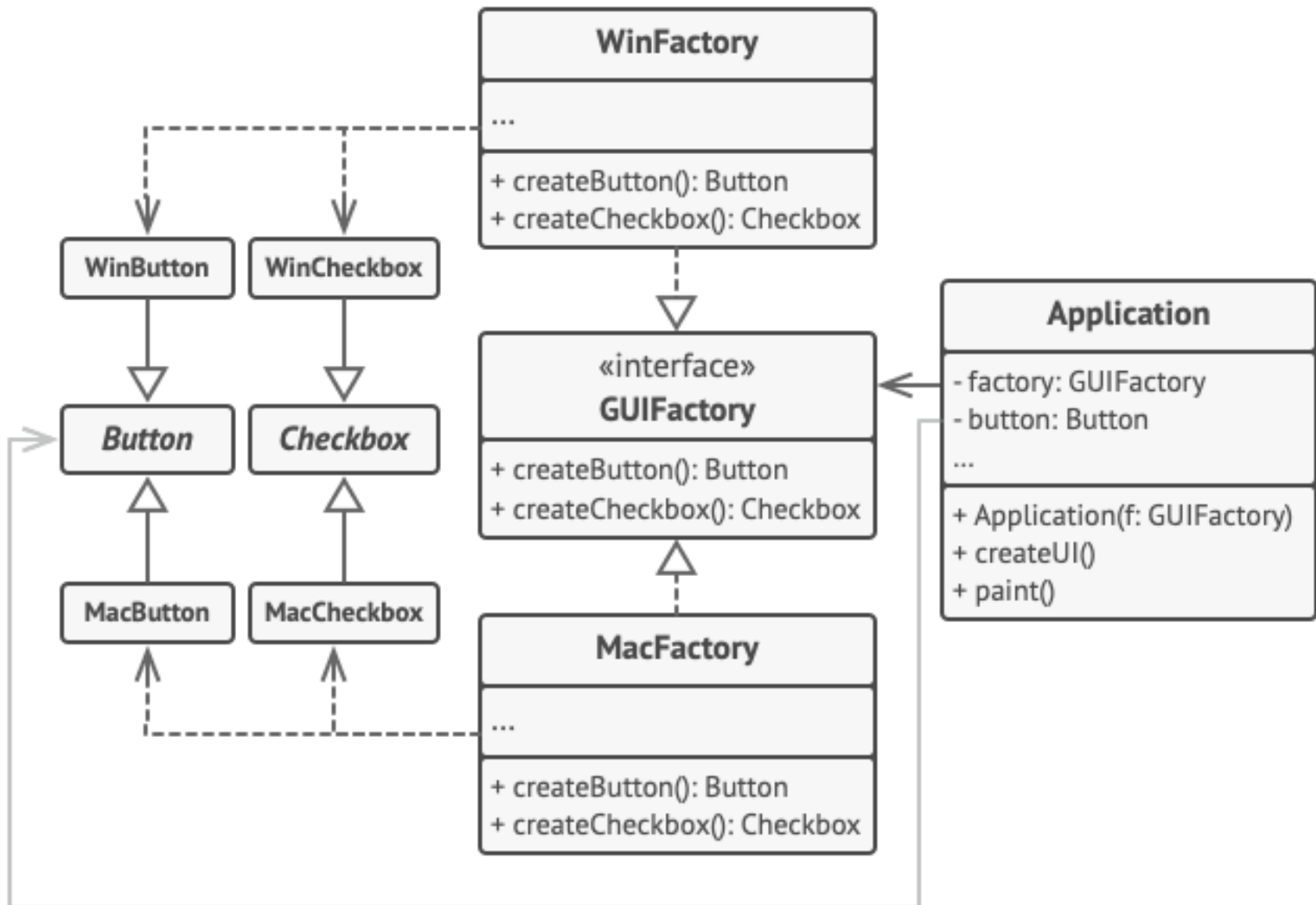
 Nói dễ hiểu:

“Chọn 1 Factory, là chọn cả hệ sinh thái object đi kèm ”

6 Cấu trúc Abstract Factory



7 Ví dụ Abstract Factory



8 Ví dụ Abtrac Factory trong Order Management

Bối cảnh Order Management. Hệ thống có thể hoạt động với nhiều thị trường với các quy định khác về:

- Cổng thanh toán (Payment gateway)
- Mức thuế
- Mẫu hóa đơn
- ...

Bước 1: Định nghĩa các Product Interface

```
public interface IPaymentService {  
    void Pay(decimal amount);  
}
```

```
public interface ITaxService {  
    decimal CalculateTax(decimal amount);  
}
```

```
public interface IInvoiceService {  
    void GenerateInvoice(Order order);  
}
```

Bước 2: Abstract Factory Interface

📌 Interface này thuộc Application Layer. Không biết VN hay US.

```
public interface IOrderServiceFactory
{
    IPaymentService CreatePaymentService();
    ITaxService CreateTaxService();
    IInvoiceService CreateInvoiceService();
}
```

Bước 3: Concrete Factories

✚ Đảm bảo tính đồng bộ: Không thể mix sai hệ VN / US.

VN Market

```
public class VNOrderServiceFactory : IOrderServiceFactory {  
    public IPaymentService CreatePaymentService() => new  
    VNPAYPaymentService();  
    public ITaxService CreateTaxService() => new  
    VATTaxService();  
    public IInvoiceService CreateInvoiceService() => new  
    VATInvoiceService();  
}
```

US Market

```
public class USOrderServiceFactory : IOrderServiceFactory {  
    public IPaymentService CreatePaymentService() => new  
    PaypalPaymentService();  
    public ITaxService CreateTaxService() => new  
    SalesTaxService();  
    public IInvoiceService CreateInvoiceService() => new  
    USInvoiceService();  
}
```

Bước 4: OrderService – Orchestrator

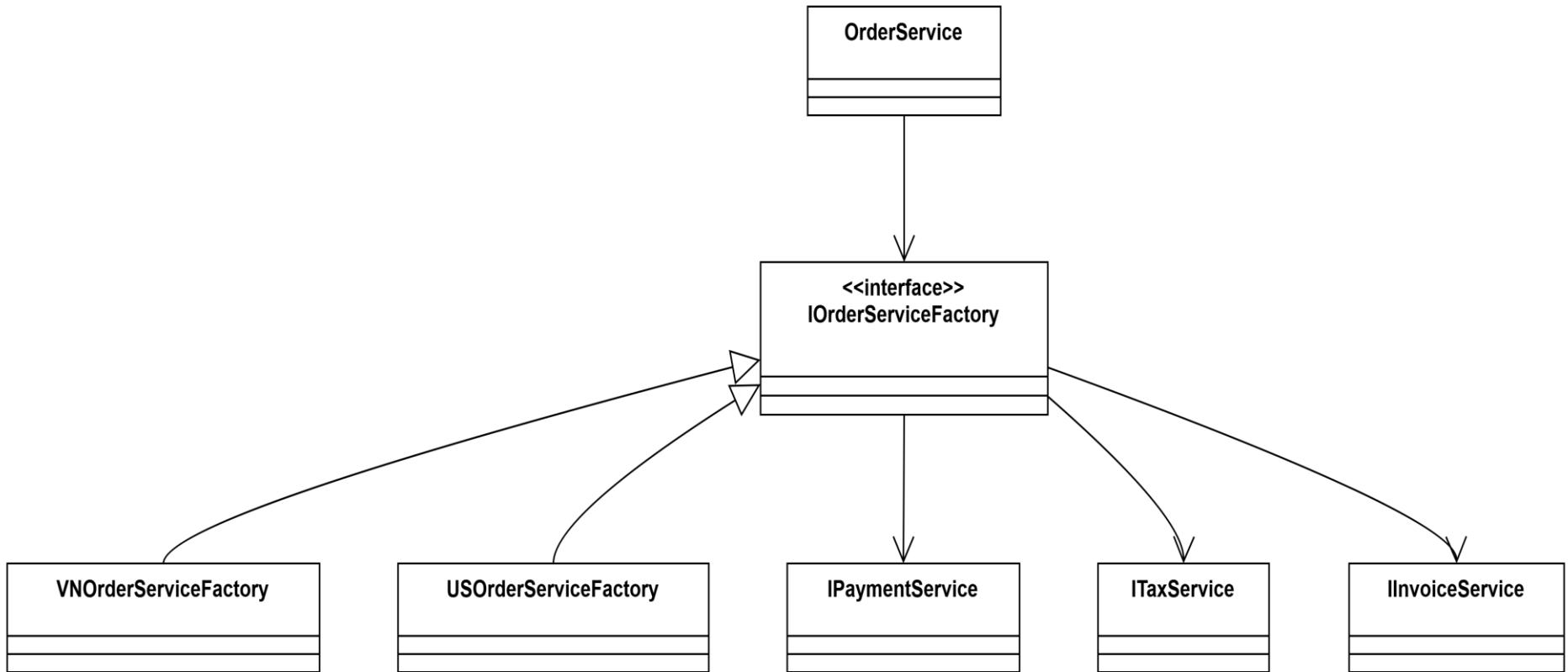
✚ OrderService sạch sẽ, không new object, không if/else region.

```
public class OrderService {
    private readonly IPaymentService _payment;
    private readonly ITaxService _tax;
    private readonly IInvoiceService _invoice;

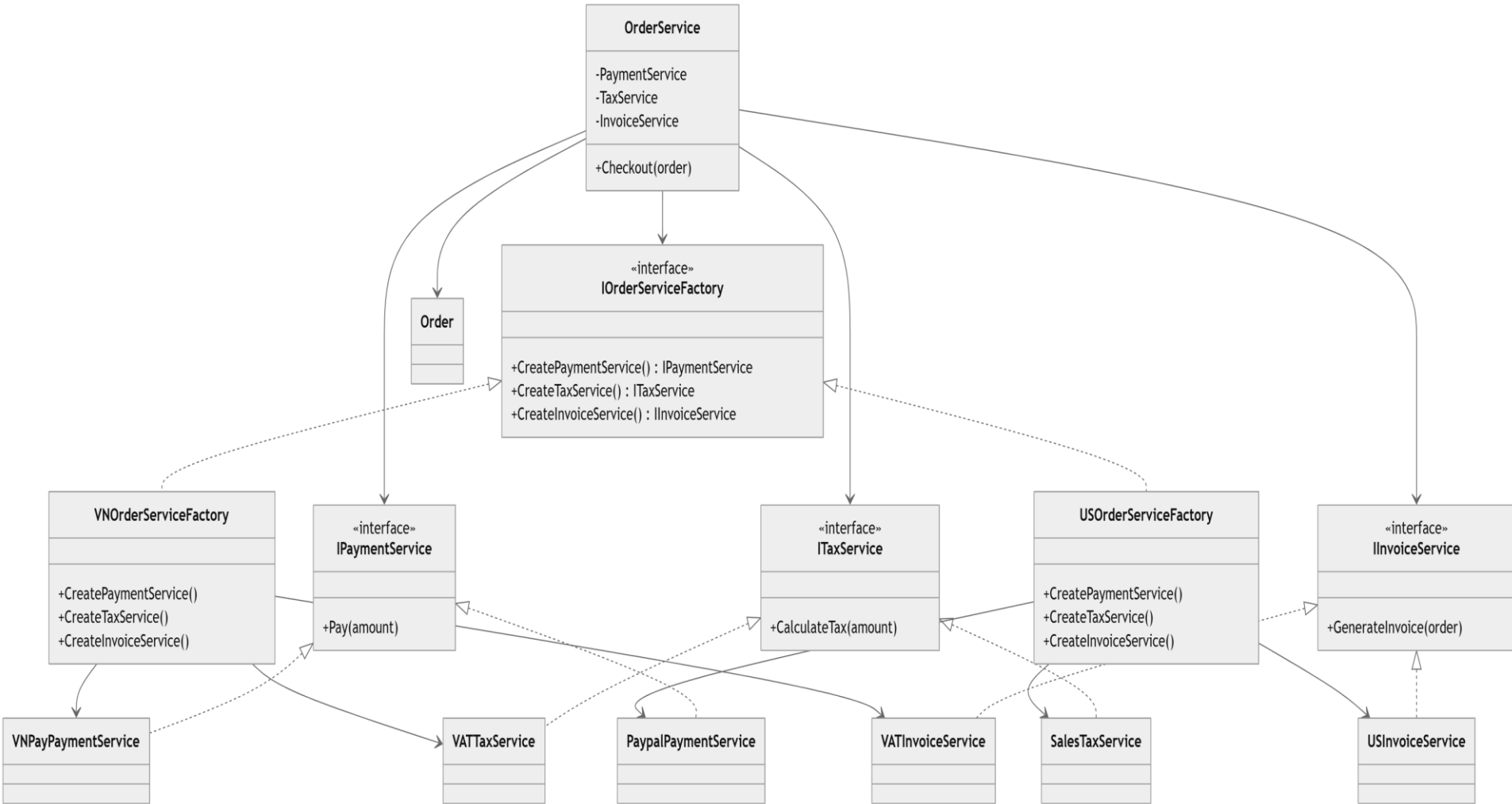
    public OrderService(IOrderServiceFactory factory) {
        // Factory tạo ra cả bộ object tương ứng
        _payment = factory.CreatePaymentService();
        _tax = factory.CreateTaxService();
        _invoice = factory.CreateInvoiceService();
    }

    public void Checkout(Order order) {
        var tax = _tax.CalculateTax(order.Amount);
        order.TotalAmount = order.Amount + tax;
        _payment.Pay(order.TotalAmount);
        _invoice.GenerateInvoice(order);
    }
}
```

9 UML class Abstract Factory trong Order Management



9 UML class Abstract Factory trong Order Management



10 Flow hoạt động runtime

Controller



OrderService



IOrderServiceFactory



VNOrderServiceFactory



VNPayPaymentService

VATTaxService

VATInvoiceService



USOrderServiceFactory



PaypalPaymentService

SalesTaxService

USInvoiceService

1 1 Mapping với GoF Pattern

GoF Role	Project class
AbstractFactory	IOrderServiceFactory
ConcreteFactory	VNOrderServiceFactory, USOrderServiceFactory
AbstractProductA	IPaymentService
AbstractProductB	ITaxService
AbstractProductC	IInvoiceService
ConcreteProduct	VNPayPaymentService, PaypalPaymentService
ConcreteProduct	VATTaxService, SalesTaxService
ConcreteProduct	VATInvoiceService, USInvoiceService
Client	OrderService

1 2 Khởi tạo và sử dụng Factory (Cách 1)

appsettings.json

```
{  
  "Market": "VN"  
}
```

Program.cs

```
IOrderServiceFactory factory;  
  
// Config hoặc Runtime decision  
  
var market = builder.Configuration["Market"];  
  
if (market == "VN")  
    factory = new VNOrderServiceFactory();  
else  
    factory = new USOrderServiceFactory();  
  
// Inject factory vào service (OrderService.cs)  
var orderService = new OrderService(factory);  
orderService.Checkout(new Order());
```

*Trong thực tế không dùng new để tạo đối tượng Factory, thường dùng các **DI Container** (.NET Built-in DI) để quản lý vòng đời của các factory này.*

1 2 Khởi tạo và sử dụng Factory (Cách 2)

Dùng DI Container (chuẩn ASP.NET Core)

```
using PaymentFactoryDemo.Services;
using PaymentFactoryDemo.Services.AbstractFactory;
var builder = WebApplication.CreateBuilder(args);
var market = builder.Configuration["Market"];
if (market == "VN"){
    builder.Services.AddScoped<IOrderServiceFactory, VNOrderServiceFactory>();
}
else{
    builder.Services.AddScoped<IOrderServiceFactory, USOrderServiceFactory>();
}
builder.Services.AddScoped<OrderService>();
builder.Services.AddControllers();
var app = builder.Build();
app.MapControllers();
app.Run();
```

1 2 Khởi tạo và sử dụng Factory (Cách 3)

Dùng DI Container (chuẩn ASP.NET Core)

```
using PaymentFactoryDemo.Services;
using PaymentFactoryDemo.Services.AbstractFactory;
var builder = WebApplication.CreateBuilder(args);
var market = builder.Configuration["Market"];
if (market == "VN"){
    builder.Services.AddScoped<IOrderServiceFactory, VNOrderServiceFactory>();
}
else{
    builder.Services.AddScoped<IOrderServiceFactory, USOrderServiceFactory>();
}
builder.Services.AddScoped<OrderService>();
builder.Services.AddControllers();
var app = builder.Build();
app.MapControllers();
app.Run();
```

1 3 DIP & Khi nào dùng?

Abstract Factory & DIP

Business phụ thuộc abstraction (Interface Factory).

Infrastructure phụ thuộc interface.

Dependency đảo chiều hoàn toàn.

→ Abstract Factory là DIP ở mức hệ thống.

Khi nào NÊN dùng?

- ✓ Hệ thống có nhiều variant / market / platform.
- ✓ Các object phải đồng bộ (Family).
- ✓ Yêu cầu mở rộng lâu dài.

1 **4** So sánh: Factory vs Abstract Factory

Tiêu chí	Factory Method	Abstract Factory
Phạm vi	Tạo 1 object	Tạo cả họ object
Độ phức tạp	Đơn giản	Phức tạp hơn
Số lượng Interface	Ít	Nhiều
Ứng dụng	Module nhỏ, độc lập	Hệ thống lớn, đa nền tảng

1 5 Bài tập trên lớp



Bài 1:

Thiết kế Abstract Factory cho: `OnlineOrder` vs `InStoreOrder`.



Bài 2:

Thêm Market EU (Euro, VAT EU, Stripe)

→ Yêu cầu: Không sửa class `OrderService`.



Bài 3:

Vẽ UML Before (Simple Factory) / After (Abstract Factory).

1 6 Câu hỏi

1. Vì sao Abstract Factory tránh được mix object sai hệ?
2. Abstract Factory có vi phạm OCP không?
3. Abstract Factory có thay thế DI không?
4. Khi nào Factory trở nên “over-engineering”?

1 7 Kết luận

“ Factory tạo object ”

“ Abstract Factory tạo hệ sinh thái ”

“ Business không nên biết mình đang sống ở hệ nào ”